CKA and CKS exam practice

Task-based practice for the hands-on Kubernetes certifications: speed setup, imperative kubectl, cluster tasks, troubleshooting and the security domains, ending with a timed mock run.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The exam environment, speed setup and the docs (13 min)
2. Imperative kubectl and fast YAML (13 min)
3. CKA: cluster setup with kubeadm and upgrades (14 min)
4. CKA: etcd backup, restore and static pods (13 min)
5. CKA: RBAC, kubeconfig and certificate tasks (13 min)
6. CKA: workloads, scheduling and configuration tasks (14 min)
7. CKA: services, Ingress, Gateway API and DNS tasks (14 min)
8. CKA: storage tasks (12 min)
9. CKA: troubleshooting method and scenarios (15 min)
10. CKS: cluster setup and hardening (14 min)
11. CKS: system hardening with AppArmor, seccomp and the host (13 min)
12. CKS: minimize microservice vulnerabilities (14 min)
13. CKS: supply chain security (13 min)
14. CKS: monitoring, runtime security and the mock run (15 min)
Cheat sheet

Lesson 1. The exam environment, speed setup and the docs

13 minutes. Prepare for the hands-on CKA and CKS format: what the browser terminal gives you, the first two minutes of setup, switching contexts safely, and finding answers in the allowed documentation fast.

The CKA and CKS are performance-based: you work in a remote terminal on real clusters, solve tasks, and are scored on the end state. The earlier lesson on exam strategy in the Kubernetes module covers what each exam tests. This lesson is about the working environment itself, because many candidates lose minutes to habits that a short setup would remove.

What you will learn

- What the exam environment is like and how scoring works
- A two-minute setup of aliases, completion and editor settings
- How to switch contexts and never work on the wrong cluster
- How to search the documentation quickly

How the exam works

Check the current candidate handbook on the Linux Foundation site before you book, because duration, passing score, Kubernetes version and allowed resources change over time. At the time of writing, each exam is about two hours, made up of separate tasks with different weights, and you must reach a passing score of around two thirds. Tasks run on several clusters, and each task tells you which context to use. You may open one extra browser tab for the official documentation, and for the CKS the documentation of a few named tools. You may not use outside notes or your own search engine.

Because scoring checks the end state, partial work counts on many tasks. That is why order matters: take the quick, high-value tasks first and flag the slow ones.

The domains and their weights

The curriculum is published by the CNCF and weights change from time to time. Use the current numbers from the official curriculum; the ones below are typical and give the shape.

CKA domain	Typical weight
Troubleshooting	30 percent
Cluster architecture, installation and configuration	25 percent
Services and networking	20 percent
Workloads and scheduling	15 percent
Storage	10 percent

CKS domain	Typical weight
Minimize microservice vulnerabilities	20 percent
Supply chain security	20 percent
Monitoring, logging and runtime security	20 percent
Cluster hardening	15 percent
System hardening	15 percent
Cluster setup	10 percent

The practical lesson of the weights: troubleshooting alone is nearly a third of the CKA, so spend a corresponding share of your practice time on it.

MARGA by EknathaLabs marga.eknathalabs.com

The two-minute setup

The terminal usually has `kubectl` completion and an alias already available, but do not assume. Check, and add what is missing. A typical setup:

```
alias k=kubectl
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
export do="--dry-run=client -o yaml"
export now="--force --grace-period=0"
```

Then set up the editor. For vim, a `~/.vimrc` with these lines saves a lot of YAML pain:

```
cat > ~/.vimrc <<'VIMRC'
set expandtab
set tabstop=2
set shiftwidth=2
set number
VIMRC
```

Spaces instead of tabs matters because YAML rejects tab indentation. Practise until typing `k run web --image=nginx $do > pod.yaml` is automatic.

Contexts: the most expensive mistake

Every task starts with a command that selects the cluster, such as `kubectl config use-context k8s-c2`. Run it first, every time. Doing a correct task on the wrong cluster scores zero and may break another task. Check with `kubectl config current-context` before long edits. Notice also that some tasks tell you to connect to a node with `ssh`; work there only as long as the task needs, then exit back to the base terminal before starting the next task.

Finding answers in the docs

The allowed documentation is the Kubernetes site, and its search works well if you give it the resource name. Habits that help:

- Search for the task keyword plus the object, for example "persistentvolume" or "network policy"
- Use the "Tasks" pages, which are written as copy-paste examples
- Copy YAML from the docs and edit it, rather than typing from memory
- Use `kubectl explain pod.spec.containers --recursive` offline in the terminal for field names; it is faster than searching

Time budget

Set a rule such as: no task takes more than its share of the clock. If a task is worth 4 percent and you have spent six minutes, flag it and move on. Return at the end. Keep a short list of flagged tasks in a notes box if the interface provides one. Leave time at the end to re-read the question text of each flagged task, because a missed requirement such as a specific file path is easy to fix when you reread.

```
tasks = [
    ("create pod with label", 2, 3),
    ("etcd backup", 7, 8),
    ("fix kubelet", 8, 12),
    ("network policy", 5, 7),
]
minutes_left = 120
for name, weight, est in sorted(tasks, key=lambda t: t[2] / t[1]):
    print(f"{name:24} weight {weight}  est {est} min  minutes per point {est / weight:.1f}")
```

Order by minutes per point and you will finish easy points first. This is a planning aid for practice, not something to run in the exam.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: First minutes of an exam task

Part	What it does	What goes wrong
Select context	Run the use-context command the task gives.	Working on the wrong cluster.
Read the whole task	Note names, paths and namespaces.	Missing a path or flag in the text.
Generate or copy YAML	Imperative commands and docs examples.	Typing long YAML from memory.
Apply and verify	Check with get and describe.	Not verifying the end state.
Flag if slow	Move on, come back later.	Sinking time into one task.

Flow: Select context to Read the whole task / Read the whole task to Generate or copy YAML / Generate or copy YAML to Apply and verify / Apply and verify to Flag if slow

Incident: The right answer on the wrong cluster

> **Incident:** *A candidate spent twelve minutes fixing a broken node, verified that the fix worked, and scored nothing for the task.*

```
$ kubectl config current-context
k8s-c1
$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
c1-cp1     Ready    control-plane   40d   v1.31.0
(this is a simulated illustration, not a real incident)
```

The task had said to use context `k8s-c3`. The candidate had skipped the `use-context` line and worked on the cluster from the previous task. The fix was correct but applied to the wrong cluster, and the broken node on the right cluster stayed broken. The habit that prevents this is to paste the context command from the task text before doing anything else.

Common mistakes

- Skipping the context command at the start of each task
- Typing long YAML from scratch instead of generating or copying it
- Using tabs in YAML
- Spending too long on one low-value task
- Forgetting to leave a node or ssh session after a task

Check yourself

What should you do at the start of every exam task?

Answer: Run the context command that the task gives

Why does the end state matter in a performance-based exam?

Answer: Tasks are scored by checking what is actually configured in the cluster

Which method is quickest for finding a field name offline?

Answer: kubectl explain with the resource path

What is a good approach to a task that is taking too long?

Answer: Flag it, move on and return if time remains

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. Imperative kubectl and fast YAML

13 minutes. Create and change resources in seconds with imperative commands, dry-run output you edit, JSONPath and custom columns, so the terminal does the typing during exam tasks and daily work.

Declarative YAML is the right way to manage clusters over time. In an exam, and in many incident situations, speed matters more. kubectl can create most common objects imperatively and print the YAML for you to adjust. Learning this set of commands is the best time investment for a hands-on test.

What you will learn

- Imperative commands for the common objects
- The dry-run pattern for generating YAML
- Quick edits, patches and set commands
- Output formats that answer questions fast

Creating objects imperatively

```
kubectl run web --image=nginx:1.27 --port=80 --labels=app=web
kubectl create deployment api --image=nginx:1.27 --replicas=3
kubectl expose deployment api --port=80 --target-port=8080 --type=ClusterIP
kubectl create configmap app-config --from-literal=mode=prod --from-file=settings.ini
kubectl create secret generic db-pass --from-literal=password=changeme
kubectl create job once --image=busybox:1.36 -- echo hello
kubectl create cronjob tick --image=busybox:1.36 --schedule="*/5 * * * *" -- date
kubectl create namespace dev
kubectl create serviceaccount deployer -n dev
```

Anything after `--` is the container command. Pin image tags, as in real life.

The dry-run pattern

When you need a field that no flag provides, generate YAML and edit it:

```
kubectl create deployment api --image=nginx:1.27 --replicas=3 --dry-run=client -o yaml > api.yaml
vim api.yaml
kubectl apply -f api.yaml
```

`--dry-run=client` prints the object without sending it to the server. With `-o yaml` you get a valid starting file. Add what you need, such as a volume or a security context, and apply. Use `--dry-run=server` when you want the API server's validation without creating anything.

Changing what exists

```
kubectl set image deployment/api nginx=nginx:1.27.1
kubectl set resources deployment/api --limits=cpu=500m,memory=256Mi --requests=cpu=100m,memory=128 \
    Mi
kubectl scale deployment/api --replicas=5
kubectl label pod web tier=frontend
kubectl annotate pod web owner=team-a
kubectl rollout status deployment/api
kubectl rollout undo deployment/api
kubectl edit deployment api
kubectl patch deployment api -p '{"spec":{"replicas":2}}'
```

`kubectl edit` opens the live object in your editor; saving applies it. For fields that are immutable, such as most pod spec fields, you must delete and recreate, or use `kubectl replace --force -f file.yaml`.

MARGA by EknathaLabs marga.eknathalabs.com

Reading output quickly

```
kubectl get pods -o wide
kubectl get pods --show-labels
kubectl get pods -l app=web
kubectl get pods -A --field-selector=status.phase!=Running
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pod web -o jsonpath='{.status.podIP}'
kubectl get nodes -o custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod web
kubectl logs web --previous
kubectl explain pod.spec.containers.securityContext
```

JSONPath and custom columns answer tasks such as "write the names of all nodes to a file" without manual copying. Redirect to a file with `>` and check it with `cat` afterwards.

Working inside pods

```
kubectl exec -it web -- sh
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://api
kubectl cp web:/etc/nginx/nginx.conf ./nginx.conf
kubectl port-forward svc/api 8080:80
```

The temporary pod pattern with `--rm -it --restart=Never` is the quickest way to test service names and network policies from inside the cluster.

Know your limits

Imperative commands cover the common cases. Pod volumes, init containers, probes, affinity, tolerations and most security settings need YAML. Know the dry-run pattern well enough that adding a block to generated YAML takes seconds. Keep a mental library of the three or four YAML snippets you use most, and copy the rest from the docs.

A quick sanity check

Before applying a generated file, read it once. Check the namespace, the name, labels matching selectors, and that no leftover fields such as `status: {}` or `creationTimestamp: null` cause problems. Those two are harmless, but a wrong namespace is not.

```
import json

# check that a generated deployment's selector matches its pod template labels
d = json.loads("""{
 "spec": {"selector": {"matchLabels": {"app": "api"}},
          "template": {"metadata": {"labels": {"app": "api", "tier": "back"}}}}
}""")
sel = d["spec"]["selector"]["matchLabels"]
labels = d["spec"]["template"]["metadata"]["labels"]
print("selector matches template:", all(labels.get(k) == v for k, v in sel.items()))
```

Diagram: From command to resource

Part	What it does	What goes wrong
kubectl create	Imperative command with flags.	Missing flags for the field you need.
Dry run output	YAML printed, nothing created.	Applying without reading.
Edit the file	Add volumes, probes or security settings.	Tabs or wrong indentation.
API server	Validates and stores the object.	Immutable field changes rejected.
Verify	get, describe and logs.	Assuming success.

Flow: kubectl create to Dry run output / Dry run output to Edit the file / Edit the file to API server / API server to Verify

Incident: The selector that did not match

Incident: *A candidate hand-edited generated YAML and applied it; the deployment never created pods.*

```
$ kubectl get deployment api
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    0/3     0            0           2m
$ kubectl describe deployment api
Selector:  app=api
Pod Template Labels:  app=backend
(this is a simulated illustration, not a real incident)
```

The pod template labels had been changed to `app=backend` while the selector still said `app=api`, so the deployment could not claim its pods. Reading the describe output revealed the mismatch within seconds. Generating YAML and changing labels in only one place is a classic slip; check both.

Common mistakes

- Typing full YAML when an imperative command and dry run would do
- Forgetting `--dry-run=client -o yaml` and creating the object directly
- Mismatching selector and pod template labels
- Applying without reading generated YAML
- Using latest tags in examples that then differ from the task's requirements

Check yourself

What does kubectl create deployment api --image=nginx --dry-run=client -o yaml do?

Answer: Prints the deployment YAML without creating it

Which command shows a pod's IP using JSONPath?

Answer: kubectl get pod web -o jsonpath='{.status.podIP}'

How do you test a service from inside the cluster quickly?

Answer: Run a temporary pod with kubectl run --rm -it --restart=Never

Why must a deployment's selector match its pod template labels?

Answer: Otherwise the deployment cannot claim or manage its pods

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. CKA: cluster setup with kubeadm and upgrades

14 minutes. Practise the cluster architecture tasks: prepare nodes, bootstrap a control plane and join workers with kubeadm, install a network plugin, upgrade a cluster one minor version, and manage nodes safely.

Cluster architecture, installation and configuration is about a quarter of the CKA. The tasks are procedural and well documented, which makes them good points for a prepared candidate. The Kubernetes module covers the concepts of the control plane and upgrades. This lesson turns them into the exact sequences you type, and what to check after each.

What you will learn

- Preparing nodes and bootstrapping with kubeadm
- Joining workers and installing a pod network
- Upgrading a cluster by one minor version
- Draining, cordoning and basic high availability concepts

Prepare the nodes

kubeadm expects a Linux host with a container runtime, the kubelet, kubeadm and kubectl installed. Before initialising, the usual preparation is: swap disabled, required kernel modules loaded, sysctl values for bridged traffic and IP forwarding set, unique hostnames and MAC addresses, and open ports between nodes. A task may give you nodes that are partly prepared; check each of these rather than assuming.

```
swapoff -a
lsmod | grep br_netfilter
sysctl net.ipv4.ip_forward
systemctl status containerd
```

Bootstrap the control plane

```
kubeadm init --pod-network-cidr=10.244.0.0/16 --kubernetes-version=v1.31.0
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
kubectl get nodes
```

The pod network range must match what your network plugin expects. After init, the node shows `NotReady` until a network plugin is installed. Apply the plugin's manifest following its documentation. Then check `kubectl get pods -n kube-system` until system pods are running.

Join workers

The init output prints a join command with a token and a CA certificate hash. If it has expired or you lost it, create a new one on the control plane:

```
kubeadm token create --print-join-command
```

Run the printed command as root on the worker. Then `kubectl get nodes` on the control plane should list it, and it becomes `Ready` once the network plugin is running there. For a highly available control plane, kubeadm supports additional control plane nodes behind a stable endpoint, using `--control-plane-endpoint` at init and `--control-plane` on join. Know the idea, and follow the docs for the exact flags.

Upgrade one minor version

Kubernetes supports upgrading one minor version at a time. Never skip a minor version. The documented order is control plane first, then workers, node by node. On the first control plane node:

```
apt-get update
apt-cache madison kubeadm | head
apt-mark unhold kubeadm
apt-get install -y kubeadm='1.31.1-*'
apt-mark hold kubeadm
```

MARGA by EknathaLabs marga.eknathalabs.com

```
kubeadm version
kubeadm upgrade plan
kubeadm upgrade apply v1.31.1
```

Then upgrade the kubelet and kubectl on that node, after draining it:

```
kubectl drain cp1 --ignore-daemonsets
apt-mark unhold kubelet kubectl
apt-get install -y kubelet='1.31.1-*' kubectl='1.31.1-*'
apt-mark hold kubelet kubectl
systemctl daemon-reload
systemctl restart kubelet
kubectl uncordon cp1
```

On each worker, run `kubeadm upgrade node` after installing the new kubeadm, then drain, upgrade the kubelet, restart it and uncordon. Version numbers above are only an example; use the versions given in the task. On extra control plane nodes, use `kubeadm upgrade node` instead of `apply`.

Drain, cordon and uncordon

`kubectl cordon` marks a node unschedulable but leaves running pods. `kubectl drain` cordons and also evicts pods, respecting disruption budgets; it needs `--ignore-daemonsets` because daemon set pods cannot be evicted, and `--delete-emptydir-data` when pods use emptyDir volumes. `kubectl uncordon` makes the node schedulable again. Remember to uncordon after maintenance; leaving a node cordoned is a common lost point.

Other configuration tasks

The curriculum also covers managing role-based access (next lessons), using Helm and Kustomize to install cluster components, understanding extension interfaces such as CNI, CSI and CRI, and working with custom resource definitions and operators. Practise a basic Helm install and upgrade and a Kustomize overlay, because short tasks appear around them.

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install web bitnami/nginx -n web --create-namespace --set replicaCount=2
helm list -A
kubectl kustomize ./overlays/prod
kubectl apply -k ./overlays/prod
```

Check what you did

After any cluster change, verify: nodes Ready, system pods Running, and a test workload schedules and can reach a service. Certificates created by kubeadm expire after a year unless upgraded, and `kubeadm certs check-expiration` lists them.

```
nodes = {"cp1": "1.31.1", "w1": "1.31.0", "w2": "1.31.0"}
target = "1.31.1"
todo = [n for n, v in nodes.items() if v != target]
print("nodes still to upgrade:", todo)
# upgrade order: control plane first, then workers one at a time
order = ["cp1", "w1", "w2"]
print([n for n in order if n in todo])
```

Diagram: Upgrading one minor version

Part	What it does	What goes wrong
kubeadm upgrade plan	Shows available versions and checks.	Planning a jump across minors.
Control plane node	upgrade apply, then kubelet.	Skipping the drain.
Drain a worker	Evict pods, respect budgets.	Forgetting ignore-daemonsets.
Upgrade worker	upgrade node, kubelet restart.	Wrong package version.
Uncordon	Make it schedulable again.	Left cordoned.
Verify	Nodes Ready, pods Running.	No post-check.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: kubeadm upgrade plan to Control plane node / Control plane node to Drain a worker / Drain a worker to Upgrade worker / Upgrade worker to Uncordon / Uncordon to Verify

Incident: The worker that stayed cordoned

Incident: *After an upgrade task, a worker node stayed unschedulable, and a later task's pods stayed Pending.*

```
$ kubectl get nodes
NAME   STATUS                     ROLES           AGE   VERSION
cp1    Ready                      control-plane   30d   v1.31.1
w1     Ready,SchedulingDisabled   <none>          30d   v1.31.1
(this is a simulated illustration, not a real incident)
```

The candidate had drained the worker, upgraded it, and forgot the final uncordon. Pods that needed to schedule had nowhere to go. The status text `SchedulingDisabled` is the clue. A habit of ending every maintenance sequence with `kubectl get nodes` catches it.

Common mistakes

- Skipping a minor version during an upgrade
- Forgetting the daemon set flag when draining
- Not uncordoning after maintenance
- Using a join token that has expired
- Applying a network plugin whose pod range differs from the init setting

Check yourself

In what order should a cluster be upgraded?

Answer: Control plane first, then workers, one minor version at a time

How do you get a fresh worker join command?

Answer: kubeadm token create --print-join-command

What does kubectl drain do beyond cordon?

Answer: It evicts the pods from the node

Why does a new kubeadm node stay NotReady?

Answer: A pod network plugin has not been installed or is not running

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. CKA: etcd backup, restore and static pods

13 minutes. Take an etcd snapshot with the right certificates, restore it to a new data directory and repoint the static pod, and understand static pod manifests well enough to repair a control plane.

etcd holds the cluster's state, so a backup and restore task is a classic. It is mechanical and has several exact parameters, which is why it rewards practice. The Kubernetes operations module explains why etcd matters. Here is the working sequence.

What you will learn

- Finding the etcd endpoint and certificate paths
- Taking and verifying a snapshot
- Restoring to a new directory and updating the manifest
- How static pods work and how to fix one

Find the parameters

On a kubeadm cluster, etcd runs as a static pod. Its manifest is in the kubelet's static pod directory, usually `/etc/kubernetes/manifests/etcd.yaml`. Read it to find the flags you need:

```
grep -E "listen-client-urls|cert-file|key-file|trusted-ca-file|data-dir" /etc/kubernetes/manifests \
    /etcd.yaml
```

You need the client endpoint, the CA certificate, the server certificate and its key, and the data directory. On a typical setup the certificates are under `/etc/kubernetes/pki/etcd/`.

Take a snapshot

```
ETCDCTL_API=3 etcdctl snapshot save /opt/backup/etcd-snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Check it:

```
etcdutl snapshot status /opt/backup/etcd-snapshot.db --write-out=table
```

Newer etcd releases moved offline snapshot operations to `etcdutl`; older ones used `etcdctl snapshot status` and `restore`. Run `--help` to see which your environment provides. If `etcdctl` is not installed on the host, the task may expect you to run it from the etcd container or to install it. Read the task text for hints about where to store the file, and confirm the file exists afterwards.

Restore to a new directory

Restoring does not overwrite the live data. It writes a new data directory, and you then point etcd at it.

```
etcdutl snapshot restore /opt/backup/etcd-snapshot.db --data-dir=/var/lib/etcd-restored
```

Edit the static pod manifest so the etcd data volume points to the new directory. In `/etc/kubernetes/manifests/etcd.yaml`, find the `hostPath` for the volume named `etcd-data` and change `path` to `/var/lib/etcd-restored`. When you save the manifest, the kubelet notices the change and recreates the etcd pod. Wait a minute, then check:

```
crictl ps | grep etcd
kubectl get pods -n kube-system
kubectl get nodes
```

While etcd restarts, `kubectl` will fail briefly because the API server cannot reach its store. That is normal. If it does not recover, look at the etcd container logs with `crictl logs`.

MARGA by EknathaLabs marga.eknathalabs.com

Static pods

A static pod is managed directly by the kubelet from a manifest file, not through the API server. The kubelet watches a directory, set by `staticPodPath` in its configuration file, usually `/var/lib/kubelet/config.yaml`. Control plane components on a kubeadm cluster are static pods: etcd, kube-apiserver, kube-controller-manager and kube-scheduler.

Consequences worth knowing:

- Editing the file changes the pod; the kubelet restarts it
- Removing the file removes the pod
- The API server shows a read-only "mirror" copy of the pod, named with the node name suffix
- You cannot delete a static pod with `kubectl delete`; it comes back. Remove or move its manifest file instead

Tasks may ask you to create a static pod on a worker. Find that node's `staticPodPath` and write a pod manifest there, which you can generate with a dry run.

```
kubectl run static-web --image=nginx:1.27 --dry-run=client -o yaml > /etc/kubernetes/manifests/sta \
    tic-web.yaml
```

Check what you changed

After a restore, verify that the cluster objects you expect are there, such as a namespace or deployment that existed at snapshot time. If the task asks you to restore a specific snapshot file, make sure you used that file and not a different one.

```
# a simple pre-restore check of the values you will need
required = ["endpoints", "cacert", "cert", "key", "snapshot-path", "new-data-dir"]
given = {"endpoints": "https://127.0.0.1:2379", "cacert": "/etc/kubernetes/pki/etcd/ca.crt",
         "cert": "/etc/kubernetes/pki/etcd/server.crt", "key": "/etc/kubernetes/pki/etcd/server.ke \
    y",
         "snapshot-path": "/opt/backup/etcd-snapshot.db"}
print("still needed:", [k for k in required if k not in given])
```

Diagram: Backup and restore of etcd

Part	What it does	What goes wrong
etcd manifest	Endpoint, certificates and data directory.	Reading the wrong certificate path.
Snapshot save	etcdctl with endpoint and certificates.	Missing certificate flags.
Snapshot file	Verified with snapshot status.	Not checked, saved to wrong path.
Restore	New data directory from the snapshot.	Overwriting live data.
Edit manifest	Point etcd-data at the new directory.	Changing the wrong volume.
kubelet recreates	Static pod restarts with restored data.	Typos stop the pod starting.

Flow: etcd manifest to Snapshot save / Snapshot save to Snapshot file / Snapshot file to Restore / Restore to Edit manifest / Edit manifest to kubelet recreates

Incident: The snapshot that was never saved

> **Incident:** *A candidate ran the backup command, saw no error and moved on. Later the restore task failed because the snapshot file was empty.*

```
$ ETCDCTL_API=3 etcdctl snapshot save /opt/backup/etcd.db --endpoints=https://127.0.0.1:2379
Error: context deadline exceeded
$ ls -l /opt/backup/etcd.db
-rw------- 1 root root 0 Nov 14 08:40 /opt/backup/etcd.db
(this is a simulated illustration, not a real incident)
```

The command had been run without the certificate flags, so it hung and timed out, leaving an empty file. The error had scrolled by. Always check the exit status or the file size, and run `snapshot status` on the result. A snapshot you have not verified is not a backup.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Omitting one of the four connection flags
- Not verifying the snapshot after saving it
- Restoring into the live data directory
- Editing the wrong volume in the etcd manifest
- Trying to delete a static pod with kubectl instead of removing its manifest

Check yourself

Where does a kubeadm cluster usually keep the control plane static pod manifests?

Answer: /etc/kubernetes/manifests

Which flags does an etcdctl snapshot save normally need on a secured cluster?

Answer: Endpoints, CA certificate, client certificate and key

After restoring a snapshot to a new directory, what must you change?

Answer: The etcd static pod manifest so its data volume points to the new directory

How do you remove a static pod?

Answer: Remove or move its manifest file from the static pod directory

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. CKA: RBAC, kubeconfig and certificate tasks

13 minutes. Create roles and bindings, test permissions with auth can-i, build kubeconfig entries and issue a user certificate through the CSR API, the access tasks that appear throughout the CKA.

Access control questions are quick to solve when you know the commands, and slow when you build the YAML from scratch. This lesson gives you the imperative forms, the checks and the certificate flow for creating a user.

What you will learn

- Roles, cluster roles and bindings from the command line
- Verifying access with auth can-i
- Service accounts and their tokens
- Creating a user certificate with the CSR API and kubeconfig

Roles and bindings

A Role grants permissions inside one namespace. A ClusterRole grants cluster-wide permissions, or permissions on cluster-scoped resources. A RoleBinding grants a Role (or a ClusterRole, within one namespace) to subjects; a ClusterRoleBinding grants cluster-wide.

```
kubectl create role pod-reader --verb=get,list,watch --resource=pods -n dev
kubectl create rolebinding read-pods --role=pod-reader --user=jane -n dev
kubectl create clusterrole node-viewer --verb=get,list --resource=nodes
kubectl create clusterrolebinding view-nodes --clusterrole=node-viewer --user=jane
kubectl create rolebinding deployer --clusterrole=edit --serviceaccount=dev:deployer -n dev
```

Note the service account form, `namespace:name`. Reusing a built-in ClusterRole such as `view`, `edit` or `admin` through a RoleBinding limits it to one namespace and saves writing rules.

Test before you trust

```
kubectl auth can-i list pods -n dev --as jane
kubectl auth can-i delete pods -n dev --as jane
kubectl auth can-i get nodes --as jane
kubectl auth can-i --list -n dev --as jane
kubectl auth can-i create deployments -n dev --as system:serviceaccount:dev:deployer
```

The `--as` option checks permissions for another identity, which needs your own account to be allowed to impersonate. Use `yes` and `no` outputs as your verification. If a task says "allow user X to do Y", confirm both the allowed action and one that should be denied.

Aggregated rules and subresources

Rules can name subresources such as `pods/log` or `deployments/scale`, and specific resource names through `--resource-name`. When a task needs "read logs of pods" you need `pods/log`, not only `pods`. If you are unsure, use `kubectl api-resources` to see resource names and API groups, and note that group names matter for the `apiGroups` field in YAML: core resources use an empty group, deployments use `apps`.

Service accounts

```
kubectl create serviceaccount deployer -n dev
kubectl create token deployer -n dev
kubectl run tool --image=busybox:1.36 --overrides='{"spec":{"serviceAccountName":"deployer"}}' -n  \
    dev --dry-run=client -o yaml
```

Since Kubernetes 1.24, service accounts no longer get long-lived token secrets automatically, and `kubectl create token` issues a short-lived token. In a pod, set `serviceAccountName` in the spec to use an account. To stop a pod from receiving a token it does not need, set `automountServiceAccountToken: false`.

MARGA by EknathaLabs marga.eknathalabs.com

Create a user with the CSR API

Kubernetes has no user objects; users are identities proven by certificates or other authentication. To add a user with a client certificate:

```
openssl genrsa -out jane.key 2048
openssl req -new -key jane.key -subj "/CN=jane/O=dev-team" -out jane.csr
base64 -w0 jane.csr
```

Create a CertificateSigningRequest object with that content:

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: jane
spec:
  request: LS0tLS1CRUdJTi...   # base64 of jane.csr
  signerName: kubernetes.io/kube-apiserver-client
  usages:
    - client auth
```

Then approve it and fetch the certificate:

```
kubectl certificate approve jane
kubectl get csr jane -o jsonpath='{.status.certificate}' | base64 -d > jane.crt
```

The CN becomes the user name and O becomes a group. Grant permissions with a binding to the user or group.

Build a kubeconfig entry

```
kubectl config set-credentials jane --client-certificate=jane.crt --client-key=jane.key --embed-ce \
    rts=true
kubectl config set-context jane@kubernetes --cluster=kubernetes --user=jane --namespace=dev
kubectl config use-context jane@kubernetes
kubectl config view --minify
```

When a task says to write a kubeconfig to a given file, use `--kubeconfig=/path` consistently on each command, or set `KUBECONFIG`. Check the result with `kubectl --kubeconfig=/path config view`.

Check bindings you did not write

Sometimes the task is to find who can do something dangerous or why a service account is denied. Use `kubectl get rolebinding,clusterrolebinding -A -o wide`, and `kubectl describe` a binding to see the role and subjects. Remember that permissions only add up: there are no deny rules.

```
rules = {
    "pod-reader": {"pods": {"get", "list", "watch"}},
    "editor": {"pods": {"get", "list", "create", "delete"}, "deployments": {"get", "list", "create \
    "}},
}
bindings = {"jane": ["pod-reader"], "deployer": ["editor"]}

def can(user, verb, resource):
    return any(verb in rules[r].get(resource, set()) for r in bindings.get(user, []))

print(can("jane", "list", "pods"), can("jane", "delete", "pods"), can("deployer", "create", "deplo \
    yments"))
```

Diagram: Giving a user access

Part	What it does	What goes wrong
Key and CSR	openssl with CN and O.	Wrong common name.
CertificateSigning Request	Submitted to the API.	Wrong signer name.
Approve and fetch	kubectl certificate approve.	Forgetting to approve.
kubeconfig entry	Credentials and context.	Wrong cluster name.
Role and binding	What the identity may do.	Binding to a different name.
auth can-i	Prove allowed and denied.	Testing only the allowed case.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Key and CSR to CertificateSigningRequest / CertificateSigningRequest to Approve and fetch / Approve and fetch to kubeconfig entry / kubeconfig entry to Role and binding / Role and binding to auth can-i

Incident: Allowed to list, not to read logs

Incident: *A user with a pod reader role could list pods but got an error when reading logs for a task.*

```
$ kubectl auth can-i get pods/log -n dev --as jane
no
$ kubectl logs web -n dev --as jane
Error from server (Forbidden): pods "web" is forbidden: User "jane" cannot get resource "pods/log" \
     in API group "" in the namespace "dev"
(this is a simulated illustration, not a real incident)
```

The role named only `pods`, but logs are a separate subresource, `pods/log`. Adding it to the role's resources fixed the problem. The error message names the exact resource, verb and namespace, which is the quickest guide to what to add.

Common mistakes

- Forgetting subresources such as pods/log or pods/exec
- Writing the service account subject without its namespace
- Creating a CSR with the wrong signer name
- Granting a ClusterRoleBinding when a RoleBinding was enough
- Testing only that access is allowed, not that other access is denied

Check yourself

Which command checks whether a user may list pods in a namespace?

Answer: kubectl auth can-i list pods -n dev --as jane

How does a RoleBinding differ from a ClusterRoleBinding?

Answer: A RoleBinding grants permissions in one namespace, a ClusterRoleBinding across the cluster

What does the CN field of a client certificate become?

Answer: The user name

Why can a user list pods but fail to read logs?

Answer: Logs are a separate subresource, pods/log

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. CKA: workloads, scheduling and configuration tasks

14 minutes. Solve the workload tasks quickly: rollouts and rollbacks, config and secrets, resource limits, scaling, and placing pods with taints, tolerations, node selectors and affinity.

Workloads and scheduling is about fifteen percent of the CKA, but the same skills appear inside other tasks, so they pay off widely. Most of these tasks combine a short imperative command with one block of YAML that you copy from the docs. The goal here is to know which block you need and where to put it.

What you will learn

- Rollouts, rollbacks and scaling
- ConfigMaps and Secrets used as environment variables and volumes
- Requests, limits and autoscaling
- Placement: node selectors, affinity, taints and tolerations

Deployments, rollouts and rollbacks

```
kubectl create deployment web --image=nginx:1.26 --replicas=3
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
kubectl rollout undo deployment/web --to-revision=1
kubectl scale deployment/web --replicas=5
```

The container name for `set image` is the name inside the pod template, which for `kubectl create deployment` equals the image base name. If unsure, `kubectl get deployment web -o jsonpath='{.spec.template.spec.containers[*].name}'` tells you. The rolling update strategy, with `maxSurge` and `maxUnavailable`, is set under `spec.strategy`; know where it lives.

ConfigMaps and Secrets

```
kubectl create configmap app --from-literal=LOG_LEVEL=debug
kubectl create secret generic db --from-literal=password=s3cret
```

Use them in a pod as environment variables or files:

```
spec:
  containers:
    - name: app
      image: nginx:1.27
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: app
              key: LOG_LEVEL
      envFrom:
        - secretRef:
            name: db
      volumeMounts:
        - name: cfg
          mountPath: /etc/app
  volumes:
    - name: cfg
      configMap:
        name: app
```

Secrets are base64 encoded, not encrypted; `kubectl get secret db -o jsonpath='{.data.password}' | base64 -d` decodes one. Mounting a ConfigMap as a volume creates a file per key, and mounted files update after a while when the ConfigMap changes, while environment variables do not.

MARGA by EknathaLabs marga.eknathalabs.com

Resources and quotas

Requests are what the scheduler reserves; limits are the ceiling at runtime. A container that exceeds its memory limit is killed with an out-of-memory status, and one that exceeds its CPU limit is throttled.

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

A LimitRange sets default and maximum values per namespace, and a ResourceQuota caps the total. A task that says "pods in this namespace must not exceed X" usually means one of these. A pod that asks for more than any node can offer stays Pending, with an event that says insufficient CPU or memory.

Autoscaling

A HorizontalPodAutoscaler needs metrics, normally from the metrics server, and CPU requests set on the target pods. A compact definition:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Check with `kubectl get hpa`. If the target shows `<unknown>`, metrics are missing or the pods have no CPU request.

Placing pods

Four tools, from simplest to most flexible:

- **nodeSelector**: match a label on the node. `kubectl label node w1 disk=ssd`, then `nodeSelector: {disk: ssd}` in the pod spec.
- **Node affinity**: required or preferred rules with operators; more expressive than a selector.
- **Taints and tolerations**: a taint repels pods from a node, and a toleration lets a pod ignore it.
- **Pod affinity and anti-affinity**: place pods near or away from other pods, using a topology key such as `kubernetes.io/hostname`.

```
kubectl taint nodes w1 dedicated=gpu:NoSchedule
kubectl taint nodes w1 dedicated=gpu:NoSchedule-
kubectl describe node w1 | grep -i taint
```

The trailing minus removes the taint. A toleration matching it looks like:

```
tolerations:
  - key: dedicated
    operator: Equal
    value: gpu
    effect: NoSchedule
```

MARGA by EknathaLabs marga.eknathalabs.com

A toleration allows scheduling on a tainted node; it does not require it. To force pods onto those nodes, combine a toleration with a node selector or affinity.

Probes, init containers and disruption

Liveness probes restart stuck containers, readiness probes control service traffic, and startup probes protect slow starters. Init containers run to completion before app containers, which suits waiting for a dependency. A PodDisruptionBudget limits voluntary evictions, such as during a drain; a budget that requires all replicas to be available blocks the drain forever.

Verify placement

After creating a pod, `kubectl get pod -o wide` shows its node. If it is Pending, `kubectl describe pod` lists the scheduler's reason, which names taints, selectors or resources.

```
nodes = {"w1": {"taints": {"dedicated=gpu"}, "labels": {"disk": "ssd"}},
         "w2": {"taints": set(), "labels": {"disk": "hdd"}}}
pod = {"tolerations": set(), "selector": {"disk": "ssd"}}

def fits(node):
    if not node["taints"] <= pod["tolerations"]:
        return "tainted"
    if any(node["labels"].get(k) != v for k, v in pod["selector"].items()):
        return "selector mismatch"
    return "ok"

print({n: fits(v) for n, v in nodes.items()})
```

Diagram: How a pod finds a node

Part	What it does	What goes wrong
Pending pod	Needs a node that fits.	No node qualifies.
Resource fit	Requests against free capacity.	Requests larger than any node.
Taints and tolerations	Tainted nodes repel unless tolerated.	Toleration missing.
Selectors and affinity	Labels the pod asks for.	Label typo on the node.
Scheduler decision	Chooses among the nodes left.	Events show why none fit.
Running pod	Bound to a node.	Verify with get -o wide.

Flow: Pending pod to Resource fit / Resource fit to Taints and tolerations / Resource fit to Selectors and affinity / Taints and tolerations to Scheduler decision / Selectors and affinity to Scheduler decision / Scheduler decision to Running pod

Incident: Pending because of a taint

Incident: *A pod stayed Pending, and the candidate kept changing resource limits instead of reading the event.*

```
$ kubectl describe pod batch-1
Events:
  Warning  FailedScheduling  0/3 nodes are available: 1 node(s) had untolerated taint {dedicated:  \
    gpu}, 2 node(s) didn't match Pod's node affinity/selector.
(this is a simulated illustration, not a real incident)
```

The event names both causes: one node was tainted and the others did not match the node selector. Reading the events line showed that the selector pointed to a label that did not exist on any untainted node. The fix was to correct the label, not to adjust resources. The scheduler's message is nearly always the shortest route to the cause.

Common mistakes

- Using the wrong container name in kubectl set image
- Expecting a toleration to force pods onto a node

MARGA by EknathaLabs marga.eknathalabs.com

- Setting CPU target in an HPA but no CPU request on the pods
- A PodDisruptionBudget that blocks draining
- Changing resources when the event points to a taint or selector

Check yourself

What does a toleration do?

Answer: It allows a pod to be scheduled on a node with a matching taint

How do you remove a taint with kubectl?

Answer: Repeat the taint with a trailing minus

Why would an HPA show unknown targets?

Answer: Metrics are unavailable or pods lack CPU requests

What is the fastest way to learn why a pod is Pending?

Answer: kubectl describe pod and read the events

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. CKA: services, Ingress, Gateway API and DNS tasks

14 minutes. Expose and secure applications: create services of each type, route HTTP with Ingress and the Gateway API, write network policies, and diagnose DNS with CoreDNS.

Services and networking is about a fifth of the CKA. The tasks are short, but the objects interact, so a missed label or port is easy to make. This lesson gives the shapes you need and a checking routine.

What you will learn

- Service types and what each is for
- Ingress and Gateway API routing
- Network policies, including default deny
- DNS names and CoreDNS checks

Services

```
kubectl expose deployment web --port=80 --target-port=8080
kubectl expose deployment web --name=web-np --port=80 --target-port=8080 --type=NodePort
kubectl get svc web
kubectl get endpoints web
kubectl get endpointslices -l kubernetes.io/service-name=web
```

A ClusterIP service gives a stable internal address. NodePort opens a port on every node. LoadBalancer asks the cloud provider for an external load balancer. A headless service, with `clusterIP: None`, returns pod addresses directly in DNS. The check that matters most is the endpoints list: if it is empty, the service selector does not match any ready pod. Compare `kubectl get svc web -o yaml` selector with pod labels.

Ingress

An Ingress routes HTTP and HTTPS traffic to services, and it needs an Ingress controller running in the cluster. The Ingress resource alone does nothing.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

Notice `pathType`, which is required, and the backend structure. You can generate a starting point with `kubectl create ingress web --rule="shop.example.com/api*=api:80" --dry-run=client -o yaml`. TLS adds a `tls` section naming a secret created with `kubectl create secret tls`.

Gateway API

The Gateway API is the newer, more expressive model, and appears in the current curriculum. It separates roles: a GatewayClass names a controller, a Gateway defines listeners, and routes such as HTTPRoute attach to a Gateway.

```
apiVersion: gateway.networking.k8s.io/v1
```

MARGA by EknathaLabs marga.eknathalabs.com

```
kind: HTTPRoute
metadata:
  name: web
spec:
  parentRefs:
    - name: main-gateway
  hostnames:
    - shop.example.com
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api
          port: 80
```

Check status with `kubectl describe httproute web`: conditions say whether it was accepted by the gateway and whether backend references resolved. The Gateway API CRDs may need to be installed; the docs show how.

Network policies

By default all pods can talk to all pods. A NetworkPolicy selects pods and allows only listed traffic once any policy selects them. Policies need a network plugin that enforces them.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

Then allow what you need:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-to-api
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: web
      ports:
        - protocol: TCP
          port: 8080
```

Two traps: egress default deny also blocks DNS, so allow port 53 to the DNS pods; and a `from` with both `namespaceSelector` and `podSelector` in the same list item means "both must match", while two separate items mean "either".

DNS and CoreDNS

Services get names like `web.shop.svc.cluster.local`, and within the same namespace `web` is enough. CoreDNS runs as a deployment in `kube-system` with a ConfigMap named `coredns`.

```
kubectl run dns --rm -it --image=busybox:1.36 --restart=Never -- nslookup web.shop
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl -n kube-system get configmap coredns -o yaml
```

MARGA by EknathaLabs marga.eknathalabs.com

If DNS fails inside pods, check that the CoreDNS pods are running, the `kube-dns` service has endpoints, and no network policy blocks port 53 traffic.

Debug from the client

When a service does not answer, test in order: does the name resolve, does the service have endpoints, does the pod answer on its own IP and port, and does a policy block the route? Use a temporary pod to curl each.

```
svc_selector = {"app": "web"}
pods = [{"name": "web-1", "labels": {"app": "web"}, "ready": True},
        {"name": "web-2", "labels": {"app": "web"}, "ready": False},
        {"name": "api-1", "labels": {"app": "api"}, "ready": True}]
endpoints = [p["name"] for p in pods if p["ready"] and all(p["labels"].get(k) == v for k, v in svc \
    _selector.items())]
print("endpoints:", endpoints)
```

Diagram: Reaching an application

Part	What it does	What goes wrong
Client pod	Resolves a name and connects.	DNS egress blocked by policy.
CoreDNS	Resolves service names.	CoreDNS pods not running.
Service	Stable address and selector.	Selector matches no pods.
Network policy	Allows or blocks the connection.	Default deny without an allow.
Ingress or route	External HTTP entry.	No controller installed.
Ready pods	Endpoints behind the service.	Failing readiness probes.

Flow: Client pod to CoreDNS / Client pod to Service / Ingress or route to Service / Network policy to Ready pods / Service to Ready pods

Incident: DNS blocked by default deny

Incident: *After adding a default deny egress policy, applications could not reach anything by name.*

```
$ kubectl exec web-1 -n shop -- nslookup api
;; connection timed out; no servers could be reached
$ kubectl get networkpolicy -n shop
NAME           POD-SELECTOR   AGE
default-deny   <none>         3m
(this is a simulated illustration, not a real incident)
```

The deny policy applied to all pods in the namespace, including their DNS lookups to CoreDNS. Adding an egress rule allowing UDP and TCP port 53 to pods in `kube-system` restored name resolution. Whenever you apply egress deny, add DNS in the same change.

Common mistakes

- Creating an Ingress with no controller installed
- Omitting pathType in an Ingress
- A service selector that does not match any pod label
- Blocking DNS with an egress default deny
- Misreading namespaceSelector and podSelector in one entry as "or"

Check yourself

What shows that a service selector matches no ready pods?

Answer: An empty endpoints list

What must exist for an Ingress resource to work?

Answer: An Ingress controller running in the cluster

MARGA by EknathaLabs marga.eknathalabs.com

What does a NetworkPolicy with an empty podSelector select?

Answer: All pods in its namespace

Why can egress default deny break applications?

Answer: It also blocks DNS lookups unless port 53 is allowed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. CKA: storage tasks

12 minutes. Handle persistent storage tasks: create volumes and claims that bind, use storage classes and dynamic provisioning, choose access modes and reclaim policies, mount volumes in pods and expand claims.

Storage is the smallest CKA domain by weight, about a tenth, yet the tasks are specific and a single wrong field stops a claim from binding. The Kubernetes module explains the model. Here you practise the objects and the failure patterns.

What you will learn

- PersistentVolumes, claims and how binding works
- StorageClasses and dynamic provisioning
- Access modes, volume modes and reclaim policies
- Mounting, expanding and debugging claims

The objects

A **PersistentVolume** (PV) is a piece of storage in the cluster. A **PersistentVolumeClaim** (PVC) is a request for storage by a user. A pod mounts a claim, not the volume directly. A **StorageClass** describes a kind of storage and how to provision it dynamically.

A static volume, backed by a directory on a node, as used in exercises:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data
```

A claim that can bind to it:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
  namespace: shop
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
  storageClassName: manual
```

Binding requires the claim's class, access modes and capacity request to be satisfied by a volume: the volume must be at least as large, include the requested access mode and match the storage class name. A claim without a storage class name uses the cluster's default class if one exists; set `storageClassName: ""` to request a volume with no class.

MARGA by EknathaLabs marga.eknathalabs.com

Use the claim in a pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app
  namespace: shop
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: data
```

The claim must be in the same namespace as the pod. Volumes themselves are cluster-scoped.

Access modes

- `ReadWriteOnce` (RWO): mounted read-write by one node
- `ReadOnlyMany` (ROX): mounted read-only by many nodes
- `ReadWriteMany` (RWX): mounted read-write by many nodes
- `ReadWriteOncePod` (RWOP): mounted read-write by one pod only

Not every storage type supports every mode, so a request for RWX on a block disk will not bind. RWO is per node, so several pods on the same node can share it.

StorageClasses and dynamic provisioning

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: example.com/provisioner
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

With dynamic provisioning, creating a claim that names the class makes the provisioner create a volume. `WaitForFirstConsumer` delays binding until a pod is scheduled, so the volume is created where the pod can use it; that is why a claim can stay `Pending` until a pod uses it, and that is not an error. A default class is marked with the annotation `storageclass.kubernetes.io/is-default-class: "true"`.

Reclaim policy

What happens to the volume when its claim is deleted:

- `Retain`: the volume stays with its data and must be cleaned manually; it shows `Released` and will not bind again until the old claim reference is cleared
- `Delete`: the volume and the backing storage are deleted
- `Recycle` is deprecated

You can change the policy on an existing PV with `kubectl patch pv pv-data -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'`.

Expanding a claim

If the class allows expansion, edit the claim and increase `spec.resources.requests.storage`. The size can only grow. Some drivers need the pod to restart before the filesystem resize completes, and the claim shows conditions describing progress.

MARGA by EknathaLabs marga.eknathalabs.com

Other volume types

Know the short list: `emptyDir` (scratch space that lives as long as the pod), `hostPath` (a path on the node, risky), `configMap` and `secret` (configuration), `projected`, and `persistentVolumeClaim`. Tasks may ask for two containers sharing an `emptyDir`.

Debug a claim

```
kubectl get pv,pvc -A
kubectl describe pvc data -n shop
kubectl get storageclass
kubectl describe pod app -n shop
```

The events on the claim and pod show what is missing: no matching volume, no class, a provisioner that is not running or a node that cannot mount.

```
pvs = [{"name": "pv-data", "size": 5, "modes": {"RWO"}, "cls": "manual", "free": True}]
claim = {"size": 2, "modes": {"RWO"}, "cls": "manual"}
match = [p["name"] for p in pvs if p["free"] and p["size"] >= claim["size"] and claim["modes"] <=  \
    p["modes"] and p["cls"] == claim["cls"]]
print("binds to:", match)
```

Diagram: From claim to mounted storage

Part	What it does	What goes wrong
StorageClass	Type of storage and provisioner.	No default class set.
PersistentVolume Claim	Size, mode and class requested.	Class name typo.
PersistentVolume	Static or dynamically created.	Too small or wrong mode.
Pod	Mounts the claim by name.	Claim in another namespace.
Container path	Data visible at the mount path.	Wrong mountPath.
Reclaim policy	Retain or delete on claim removal.	Data lost on Delete.

Flow: StorageClass to PersistentVolumeClaim / PersistentVolumeClaim to PersistentVolume / PersistentVolume to Pod / Pod to Container path / PersistentVolume to Reclaim policy

Incident: The claim that stayed Pending

> **Incident:** *A claim stayed Pending, and the candidate suspected the provisioner. A pod using it had never been created.*

```
$ kubectl get pvc data -n shop
NAME   STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
data   Pending                                      fast           6m
$ kubectl describe pvc data -n shop
Events:
  Normal  WaitForFirstConsumer  6m  persistentvolume-controller  waiting for first consumer to be  \
    created before binding
(this is a simulated illustration, not a real incident)
```

The storage class used `WaitForFirstConsumer`, so nothing bound until a pod referenced the claim. The event said so plainly. Creating the pod made the volume appear and the claim bind. Reading the events before changing anything saved time.

Common mistakes

- Requesting a size or mode no volume can satisfy
- A storage class name mismatch between PV and PVC
- Creating the claim in a different namespace from the pod
- Expecting a Pending claim to bind before any pod uses it, with WaitForFirstConsumer
- Forgetting that Retain leaves volumes Released and not reusable until cleaned

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What must be true for a claim to bind to a static volume?

Answer: Class matches, access modes are included and capacity is large enough

Why might a claim stay Pending with WaitForFirstConsumer?

Answer: Binding waits until a pod using the claim is scheduled

What does the Retain reclaim policy do after the claim is deleted?

Answer: Keeps the volume and its data until an administrator cleans it

When can a claim be expanded?

Answer: When its storage class allows volume expansion, and only to a larger size

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. CKA: troubleshooting method and scenarios

15 minutes. Troubleshooting is the largest CKA domain. Learn a fixed order of checks, then apply it to application failures, services, nodes that are NotReady, broken control plane components and certificate expiry.

Troubleshooting carries about thirty percent of the CKA, and in real life it is the skill that matters most. The exam gives you a broken thing and a time limit. The people who do well are not the ones who know the most obscure flag; they are the ones who look in the right order and read what the system tells them.

What you will learn

- A repeatable top-to-bottom order of checks
- Application and service failures
- Node failures: kubelet, runtime and network
- Control plane failures and certificates

The method

1. **Define the symptom.** What exactly fails, where, and since when?
2. **Look at the object.** `kubectl get` and `kubectl describe`, read the events.
3. **Look at the logs.** Container logs, previous logs, component logs.
4. **Go down a layer.** Pod, then node, then control plane, then infrastructure.
5. **Change one thing at a time** and verify.

Avoid guessing. Each command should answer a question you can state.

Pod failures

```
kubectl get pods -A
kubectl describe pod web-1 -n shop
kubectl logs web-1 -n shop
kubectl logs web-1 -n shop --previous
kubectl get events -n shop --sort-by=.lastTimestamp
```

Map the status to the cause:

Status	Usual cause	Where to look
Pending	No node fits, unbound claim	describe events
ImagePullBackOff	Wrong image name or tag, registry access	describe events
CrashLoopBackOff	Process exits, bad config or command	logs --previous
CreateContainerConfigError	Missing ConfigMap or Secret key	describe events
OOMKilled in last state	Memory limit too low	describe, limits
Running but not Ready	Failing readiness probe	describe, probe settings

Service failures

Work from the client to the pod: does the name resolve, does the service have endpoints, does the pod respond on the target port, and does a policy block it? The most common causes are a selector that does not match pod labels, a wrong `targetPort`, readiness probes that fail so pods are not in the endpoints, and network policies.

Node failures

When `kubectl get nodes` shows `NotReady`, log in to the node and check the kubelet and the runtime:

MARGA by EknathaLabs marga.eknathalabs.com

```
systemctl status kubelet
journalctl -u kubelet -n 50 --no-pager
systemctl status containerd
crictl ps -a
cat /var/lib/kubelet/config.yaml
```

Typical findings: the kubelet service is stopped or disabled (start and enable it), a wrong path in its configuration or service drop-in, an expired or wrong certificate, a runtime socket that does not exist, swap enabled, or a full disk. After a fix, `systemctl daemon-reload` if you changed a unit file, then `systemctl restart kubelet`, and check again from the control plane.

Control plane failures

On a kubeadm cluster the control plane components are static pods. If `kubectl` cannot connect at all, the API server is the first suspect. Look from the node:

```
crictl ps -a | grep -E "kube-apiserver|etcd|scheduler|controller"
crictl logs CONTAINER_ID
ls /etc/kubernetes/manifests
journalctl -u kubelet -n 50 --no-pager
```

Typical faults: a typo or wrong path in a manifest, a wrong certificate file name, a wrong etcd address, a port clash. The kubelet log often says that it cannot create the pod and why. When only a component such as the scheduler is down, pods stay Pending with no events from the scheduler, and `kubectl get pods -n kube-system` shows its pod failing. Fix the manifest and the kubelet recreates the pod.

If workloads start but never become Ready or deployments do not roll, look at the controller manager. If pods are created but never scheduled, look at the scheduler. Know the division.

Certificates

```
kubeadm certs check-expiration
kubeadm certs renew all
```

Expired certificates cause confusing errors such as failing `kubectl` or kubelet connections. After renewal, restart the control plane static pods by moving their manifests away and back, or restart the kubelet, as the docs describe.

Networking and DNS

Pods that cannot resolve names point to CoreDNS or a policy. Nodes that cannot reach each other point to the network plugin pods in `kube-system`. A plugin that is not running makes new pods stay in `ContainerCreating` with network errors in the events.

Resource and capacity problems

`kubectl top nodes` and `kubectl top pods` need the metrics server. A node with `MemoryPressure` or `DiskPressure` conditions evicts pods; check `kubectl describe node`.

Write down what you find

When a task asks you to record the cause, such as "write the reason the pod failed to a file", copy exact text from the events or logs rather than paraphrasing.

```
symptoms = {
    "ImagePullBackOff": "check image name, tag and registry access",
    "CrashLoopBackOff": "kubectl logs --previous",
    "Pending": "kubectl describe pod, read scheduler events",
    "NotReady node": "systemctl status kubelet and journalctl",
}
for s, hint in symptoms.items():
    print(f"{s:18} {hint}")
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A top-to-bottom troubleshooting order

Part	What it does	What goes wrong
Symptom	State what fails and where.	Fixing before understanding.
Object and events	get, describe and events.	Not reading the events.
Logs	Container and previous logs.	Missing --previous.
Node and kubelet	systemctl, journalctl and crictl.	Kubelet disabled after reboot.
Control plane	Static pod manifests and certificates.	Typo in a manifest path.
Fix and verify	One change, then check.	Changing many things at once.

Flow: Symptom to Object and events / Object and events to Logs / Logs to Node and kubelet / Logs to Control plane / Node and kubelet to Fix and verify / Control plane to Fix and verify

Incident: The kubelet that was stopped, not broken

> **Incident:** *A node was NotReady. The candidate spent ten minutes on pod networking before looking at the node itself.*

```
$ systemctl status kubelet
kubelet.service - kubelet: The Kubernetes Node Agent
     Loaded: loaded (/lib/systemd/system/kubelet.service; disabled)
     Active: inactive (dead)
(this is a simulated illustration, not a real incident)
```

The kubelet was inactive and disabled, so it had not started after a reboot. Starting and enabling it, then checking `kubectl get nodes`, fixed the task in under a minute. The lesson is to check the lowest layer a symptom could involve before digging into higher layers that depend on it.

Common mistakes

- Looking at pods when the node is down
- Forgetting logs --previous for a crashed container
- Editing several things before testing the first
- Not checking the kubelet log when a static pod does not start
- Paraphrasing an error when the task asks for the exact reason

Check yourself

Which status points to a missing ConfigMap key?

Answer: CreateContainerConfigError

Where do you look first for a node that is NotReady?

Answer: The kubelet status and logs on the node

What does CrashLoopBackOff mean?

Answer: The container keeps exiting and Kubernetes keeps restarting it with a delay

Why change only one thing at a time?

Answer: So you know which change fixed the problem and avoid new faults

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. CKS: cluster setup and hardening

14 minutes. Secure the cluster itself: network policies as a default, CIS benchmark checks, API server and kubelet flags, RBAC and service account hygiene, protecting metadata and verifying the binaries you run.

The Certified Kubernetes Security Specialist exam assumes you can already administer a cluster, which is why it requires a current CKA. Cluster setup and cluster hardening together make about a quarter of it. The idea is to take a working cluster and remove the ways it can be abused. The DevSecOps module covers the principles; here are the tasks.

What you will learn

- Default deny network policy and protecting node metadata
- Running and reading CIS benchmark results
- Hardening API server and kubelet settings
- RBAC, service accounts and verifying binaries

Default deny and metadata

Start with a default deny policy in each namespace, then allow only what is needed. A deny-all for ingress and egress was shown in the networking lesson. Cloud nodes often expose a metadata service at a link-local address, which holds credentials for the node's cloud role. Pods should usually not reach it:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-metadata
  namespace: shop
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
            except:
              - 169.254.169.254/32
```

This allows all egress except to the metadata address. Combine it with other policies as the task requires, remembering that policies add up.

CIS benchmark with kube-bench

The CIS Kubernetes Benchmark is a long list of recommended settings, and kube-bench is a tool that checks a node against it.

```
kube-bench run --targets master
kube-bench run --targets node
kube-bench run --targets master --check 1.2.1
```

The output lists each check as PASS, FAIL or WARN, with remediation text. A task usually names a failing check and asks you to fix it. The fix is almost always a flag in a static pod manifest or a kubelet configuration file. Re-run the check afterwards to confirm.

Harden the API server

Edit `/etc/kubernetes/manifests/kube-apiserver.yaml`; the kubelet restarts the pod when you save. Useful settings:

- `--anonymous-auth=false` rejects unauthenticated requests
- `--authorization-mode=Node,RBAC` enables node and role-based authorization, not `AlwaysAllow`
- `--enable-admission-plugins=NodeRestriction` limits what kubelets can change

MARGA by EknathaLabs marga.eknathalabs.com

- `--profiling=false` disables profiling endpoints
- `--audit-log-path` and related flags enable auditing (a later lesson)
- Avoid insecure ports and unneeded features

After a change, wait for the API server to return. If it does not, read `crictl logs` for the container; the cause is usually a typo. Keep a copy of the original manifest outside the manifests directory before editing, so that you can restore it quickly.

Harden the kubelet

The kubelet configuration is a file, often `/var/lib/kubelet/config.yaml`. Important settings:

```
authentication:
  anonymous:
    enabled: false
  webhook:
    enabled: true
authorization:
  mode: Webhook
readOnlyPort: 0
```

Anonymous access disabled, authorization delegated to the API server and the read-only port closed. Apply with `systemctl restart kubelet`. Etcd and the control plane should also use client certificate authentication, which kubeadm sets up by default.

RBAC and service accounts

Least privilege applies to humans and to workloads.

- Avoid `cluster-admin` bindings and wildcard verbs or resources
- Look for risky bindings with `kubectl get clusterrolebinding -o wide`
- Remove permissions that are not needed, and bind roles to groups instead of many individuals
- Set `automountServiceAccountToken: false` on pods or service accounts that do not call the API
- Use dedicated service accounts per workload, not the `default` account

```
kubectl patch serviceaccount default -n shop -p '{"automountServiceAccountToken": false}'
kubectl auth can-i --list --as system:serviceaccount:shop:default -n shop
```

Protect ingress and secrets in transit

Expose applications over TLS. Create a secret from a certificate and key, and reference it in the Ingress `tls` section:

```
kubectl create secret tls web-tls --cert=tls.crt --key=tls.key -n shop
```

Verify the binaries

A tampered binary defeats every other control. Compare the checksum of a downloaded Kubernetes binary with the published value:

```
sha512sum kubelet
echo "<expected-sha512>  kubelet" | sha512sum --check
```

A task may give you a checksum file and ask you to confirm which binary matches, or to remove the one that does not.

Keep up with versions

Running an old version leaves known vulnerabilities in place. Plan upgrades to supported versions, as practised in the earlier CKA lesson, and subscribe to security announcements.

```
checks = {
    "anonymous-auth=false": True,
    "authorization-mode includes RBAC": True,
    "NodeRestriction enabled": False,
    "kubelet read-only port closed": True,
```

MARGA by EknathaLabs marga.eknathalabs.com

```
}
for name, passed in checks.items():
    print(("PASS  " if passed else "FAIL  ") + name)
```

Diagram: Hardening a cluster

Part	What it does	What goes wrong
Benchmark	kube-bench against the CIS checks.	Treating warnings as passes.
API server flags	Anonymous off, RBAC on, NodeRestriction.	A typo takes the API down.
Kubelet config	Webhook auth, no read-only port.	Config not reloaded.
RBAC and accounts	Least privilege and no default token.	Wildcard cluster-admin.
Network policy	Default deny, block metadata.	Policies on a plugin that ignores them.
Verify and re-test	Checksums and benchmark re-run.	Trusting a binary unchecked.

Flow: Benchmark to API server flags / Benchmark to Kubelet config / API server flags to RBAC and accounts / Kubelet config to RBAC and accounts / RBAC and accounts to Network policy / Network policy to Verify and re-test

Incident: Anonymous access left on

> **Incident:** *A benchmark flagged that the kubelet accepted anonymous requests, and a quick probe from another node listed pods without credentials.*

```
$ kube-bench run --targets node --check 4.2.1
[FAIL] 4.2.1 Ensure that the --anonymous-auth argument is set to false
$ curl -sk https://10.0.0.12:10250/pods | head -c 80
{"kind":"PodList","apiVersion":"v1","metadata":{},"items":[{"metadata":{"name":"web-1"
(this is a simulated illustration, not a real incident)
```

The kubelet's configuration allowed anonymous requests and used `AlwaysAllow` authorization. Setting anonymous to false, authorization mode to Webhook and restarting the kubelet made the same request return 401. The benchmark check turned from FAIL to PASS, which is the verification the task wanted.

Common mistakes

- Editing the API server manifest without a backup copy
- Forgetting to restart the kubelet after changing its config
- Granting cluster-admin for convenience
- Leaving the default service account token mounted in pods that do not need it
- Fixing a benchmark check and not re-running it

Check yourself

What does kube-bench check?

Answer: Node and control plane settings against the CIS Kubernetes Benchmark

Which API server flag stops unauthenticated requests?

Answer: --anonymous-auth=false

Why disable automountServiceAccountToken for workloads that do not call the API?

Answer: A stolen token is useless if none is mounted

How do you check that a downloaded binary is genuine?

Answer: Compare its checksum with the published value

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. CKS: system hardening with AppArmor, seccomp and the host

13 minutes. Reduce what a compromised container or node can do: shrink the host attack surface, confine processes with AppArmor and seccomp profiles, and limit access to the kernel and network.

Containers share the host kernel. If an attacker escapes a container, the host's configuration decides how much they can reach. System hardening is the set of controls on the host and on the kernel interface that limit that damage. It is about fifteen percent of the CKS.

What you will learn

- Reducing the attack surface of the node
- Loading and applying AppArmor profiles
- Applying seccomp profiles to pods
- Other kernel and network controls

Shrink the host

A node should run what it needs and nothing more. Habits and checks:

```
ss -ltnp
systemctl list-units --type=service --state=running
dpkg -l | wc -l
systemctl disable --now snapd
apt-get remove --purge -y PACKAGE
```

Look for listening ports you do not recognise, services that are not part of the node's job, and tools that help an attacker such as compilers or network scanners. Remove or disable them. Restrict SSH: key authentication only, no root login, and limit who can connect. Remove users you do not need, and keep the node patched. A task may give you a listing and ask which unused service to stop.

AppArmor

AppArmor is a Linux security module that confines a program by a profile: which files it may read or write, which capabilities and network access it has. A profile must exist on the node where the pod runs.

```
aa-status
cat /etc/apparmor.d/k8s-deny-write
apparmor_parser -q /etc/apparmor.d/k8s-deny-write
```

A simple profile that denies writes everywhere:

```
#include <tunables/global>
profile k8s-deny-write flags=(attach_disconnected) {
  #include <abstractions/base>
  file,
  deny /** w,
}
```

Load it on every node that may run the pod, with `apparmor_parser`. Then reference it. In Kubernetes 1.30 and later, the pod field is:

```
spec:
  containers:
    - name: app
      image: nginx:1.27
      securityContext:
        appArmorProfile:
          type: Localhost
          localhostProfile: k8s-deny-write
```

Older clusters and many tutorials use an annotation instead, `container.apparmor.security.beta.kubernetes.io/app: localhost/k8s-deny-write`. Check which form the cluster version supports. If the profile is not loaded on the node, the pod fails to start, which is a useful

signal. Verify with `kubectl exec` that the restriction holds, for example that writing a file fails.

Seccomp

Seccomp filters which system calls a process may make. The default runtime profile blocks many dangerous calls while allowing normal behaviour.

```
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
```

`RuntimeDefault` applies the container runtime's default profile, which is a good baseline for almost all workloads. `Unconfined` disables filtering and should be avoided. A custom profile is a JSON file placed under the kubelet's seccomp directory, commonly `/var/lib/kubelet/seccomp`, and referenced by a path relative to it:

```
securityContext:
  seccompProfile:
    type: Localhost
    localhostProfile: profiles/audit.json
```

The profile file must exist on the node that runs the pod. A task might give you a profile and ask you to apply it to a pod, then show the pod running. If the pod stays in an error state, the events usually say the profile could not be found.

Capabilities and privileges

Containers should drop what they do not need:

```
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  runAsNonRoot: true
  capabilities:
    drop: ["ALL"]
```

Avoid `privileged: true`, host namespaces and host paths. Each one removes a layer of isolation. The pod security lesson shows how to enforce this across a namespace.

Kernel and network controls

Disable kernel modules that are not needed by blacklisting them, and keep sysctl settings tight. Limit which nodes can talk to each other and to the control plane with firewall rules or cloud security groups, and give nodes the least cloud permissions needed. A node role that can read all storage buckets turns a container escape into a data breach.

A host hardening checklist

```
checklist = {
    "no unneeded listening ports": True,
    "ssh root login disabled": True,
    "unused services stopped": False,
    "apparmor profile loaded for sensitive pods": True,
    "seccomp RuntimeDefault on pods": False,
}
print("to do:", [k for k, ok in checklist.items() if not ok])
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Layers of confinement

Part	What it does	What goes wrong
Hardened host	Few services, patched, limited SSH.	Unused services left running.
Kernel controls	Modules and sysctl limits.	Default permissive settings.
AppArmor profile	File and capability limits.	Profile not loaded on the node.
Seccomp profile	System call filtering.	Unconfined profile.
Container	Non-root, no privilege escalation.	Privileged mode.
Verify	Exec and test the restriction.	Assuming the profile applies.

Flow: Hardened host to Kernel controls / Kernel controls to AppArmor profile / Kernel controls to Seccomp profile / AppArmor profile to Container / Seccomp profile to Container / Container to Verify

Incident: The profile that was not there

> **Incident:** *A pod referencing an AppArmor profile would not start on one node, and the same manifest worked on another.*

```
$ kubectl get pod locked -o wide
NAME     READY   STATUS    RESTARTS   AGE   NODE
locked   0/1     Blocked   0          1m    w2
$ kubectl describe pod locked | grep -i apparmor
  Warning  Failed  kubelet  Cannot enforce AppArmor: profile "k8s-deny-write" is not loaded
(this is a simulated illustration, not a real incident)
```

The profile had been loaded on `w1` but not on `w2`. Loading it on every node that could schedule the pod, with a configuration management step to keep them in sync, fixed the problem. If a task has only one node, you only need to load it there; in a larger cluster, remember that the pod may land elsewhere.

Common mistakes

- Referencing an AppArmor profile that is not loaded on the node
- Loading a profile on one node and scheduling elsewhere
- Using Unconfined seccomp when RuntimeDefault would do
- Not testing that the restriction actually works
- Leaving unused services and open ports on nodes

Check yourself

What does an AppArmor profile do?

Answer: Confines a program's file access, capabilities and network use

What does the RuntimeDefault seccomp profile provide?

Answer: The container runtime's default filter of system calls

Why must an AppArmor profile be loaded on the node?

Answer: The kubelet enforces it there, and a missing profile prevents the pod from starting

Which setting reduces what a compromised container can do?

Answer: Dropping all capabilities and disabling privilege escalation

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. CKS: minimize microservice vulnerabilities

14 minutes. Make each workload a hard target: enforce pod security standards, set security contexts, sandbox untrusted code with runtime classes, encrypt secrets at rest and protect traffic between services.

This is the largest CKS domain, about a fifth of the exam. The topic is the security of the workloads themselves and the data they handle. The common thread is defence in depth: assume one layer will fail and make sure the next one holds.

What you will learn

- Enforcing Pod Security Standards with admission labels
- Security context settings that matter
- Sandboxed runtimes with RuntimeClass
- Encryption of Secrets at rest, and mutual TLS between services

Pod Security Standards and admission

Kubernetes defines three profiles. **Privileged** is unrestricted. **Baseline** blocks known privilege escalations, such as host namespaces and privileged containers. **Restricted** follows current hardening practice, requiring non-root users, dropped capabilities and a seccomp profile. The built-in Pod Security admission controller enforces them per namespace through labels:

```
kubectl label namespace shop pod-security.kubernetes.io/enforce=restricted
kubectl label namespace shop pod-security.kubernetes.io/warn=restricted pod-security.kubernetes.io \
    /audit=restricted
kubectl get namespace shop --show-labels
```

The modes are `enforce` (reject), `warn` (allow with a warning) and `audit` (log). Start with warn to see what would break. A task may ask you to label a namespace and then fix a deployment so its pods are admitted. The rejection message lists which fields violate the profile.

Security context

A restricted-compliant pod looks like this:

```
apiVersion: v1
kind: Pod
metadata:
  name: hardened
  namespace: shop
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: nginx:1.27
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: tmp
          mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
```

A read-only root filesystem breaks programs that write to disk, so give them an `emptyDir` for the paths they need. Many stock images need changes to run as non-root, for example listening on a port above

MARGA by EknathaLabs marga.eknathalabs.com

1024.

Sandboxed runtimes

Ordinary containers share the host kernel. For untrusted or multi-tenant code, a sandbox adds a second boundary, such as gVisor, which implements a user-space kernel, or Kata Containers, which uses a lightweight VM. In Kubernetes, you define a RuntimeClass and reference it from the pod.

```
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: gvisor
handler: runsc

spec:
  runtimeClassName: gvisor
  containers:
    - name: untrusted
      image: nginx:1.27
```

The handler name must match one configured in the container runtime on the node. If it does not, the pod fails with an event about the missing handler. Sandboxing has a performance cost and does not support every feature, so use it where the risk justifies it.

Secrets: handling and encryption at rest

Secrets are base64 encoded by default and stored unencrypted in etcd unless you enable encryption. Anyone with etcd access, or a backup, can read them. Enable encryption with an `EncryptionConfiguration` file and an API server flag.

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources:
      - secrets
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: <base64-encoded 32 byte key>
      - identity: {}
```

Pass `--encryption-provider-config=/etc/kubernetes/enc/enc.yaml` to the API server, and mount the file into the static pod. The first provider is used for writing; `identity` last lets the server still read old unencrypted data. Existing secrets are only encrypted when rewritten, so replace them:

```
kubectl get secrets -A -o json | kubectl replace -f -
```

Verify by reading a secret directly from etcd; the stored value should start with `k8s:enc:aescbc:v1:` instead of plain text. Production setups usually use a key management service provider so the key never sits on disk.

Also reduce exposure in use: mount secrets as files rather than environment variables when possible, limit who can read them with RBAC, and do not put them in images or configuration maps.

Traffic between services

Inside the cluster, traffic is plain unless you protect it. Mutual TLS gives each service an identity and encrypts the connection. A service mesh or a CNI with encryption features can provide it. For the exam, know the purpose and be able to read a policy that requires it, and know that network policies control who may connect while mTLS controls who is talking and hides the contents.

Admission and policy engines

Policy engines such as Open Policy Agent Gatekeeper and Kyverno enforce custom rules on objects at admission: required labels, allowed registries, banned settings. Tasks may give you a constraint template and ask you to create a constraint that applies it to a namespace, then show a violating pod being denied.

```
pod = {"runAsNonRoot": True, "allowPrivilegeEscalation": False, "readOnlyRootFilesystem": True,
```

MARGA by EknathaLabs marga.eknathalabs.com

```
       "capabilities_drop": ["ALL"], "seccomp": "RuntimeDefault", "privileged": False}
problems = []
if not pod["runAsNonRoot"]: problems.append("runs as root")
if pod["allowPrivilegeEscalation"]: problems.append("privilege escalation allowed")
if pod["capabilities_drop"] != ["ALL"]: problems.append("capabilities not dropped")
if pod["privileged"]: problems.append("privileged container")
print(problems or "meets the restricted profile")
```

Diagram: Layers around one workload

Part	What it does	What goes wrong
Admission	Pod Security labels and policy engine.	Namespace without labels.
Security context	Non-root, no escalation, dropped capabilities.	Defaults left permissive.
Runtime class	Sandbox for untrusted code.	Handler not installed on the node.
Encrypted secrets	Encryption at rest and tight RBAC.	Keys stored with the data.
Network and mTLS	Allowed paths and encrypted connections.	Plain traffic between pods.
Workload	The hardened service.	Trusting one layer only.

Flow: Admission to Security context / Security context to Runtime class / Security context to Encrypted secrets / Runtime class to Network and mTLS / Encrypted secrets to Network and mTLS / Network and mTLS to Workload

Incident: Secrets in the clear

> **Incident:** *An engineer assumed secrets were safe in the cluster. A backup of etcd was copied to a shared disk, and a quick search revealed a database password in plain text.*

```
$ ETCDCTL_API=3 etcdctl get /registry/secrets/shop/db --endpoints=https://127.0.0.1:2379 --cacert= \
    /etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernet \
    es/pki/etcd/server.key | strings | head -3
/registry/secrets/shop/db
password
s3cret
(this is a simulated illustration, not a real incident)
```

The secret was stored unencrypted. The team enabled encryption at rest, rewrote all secrets, rotated the exposed password and restricted who could read etcd backups. Reading the value back with etcdctl and seeing the `k8s:enc:` prefix confirmed the fix.

Common mistakes

- Enabling encryption but not rewriting existing secrets
- Enforcing a Pod Security profile without checking what would break first
- Setting a read-only root filesystem without writable paths for temp files
- Referencing a RuntimeClass whose handler is not installed
- Treating base64 as protection

Check yourself

What do the Pod Security Standards offer?

Answer: Three profiles, Privileged, Baseline and Restricted, enforced by namespace labels

How are existing secrets encrypted after enabling encryption at rest?

Answer: By rewriting them, for example with kubectl replace

What is the purpose of a RuntimeClass such as gVisor?

Answer: To run untrusted workloads behind an extra isolation layer

Why give a read-only root filesystem container an emptyDir?

Answer: Programs often need a writable place for temporary files

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 13. CKS: supply chain security

13 minutes. Control what runs in your cluster: minimise and scan images, pin by digest, allow only trusted registries through admission control, analyse manifests statically, and verify signatures.

Supply chain security is another fifth of the CKS. The idea is that the cluster is only as safe as the images, charts and manifests it runs. The DevSecOps module covers the principles of provenance. This lesson is about the tools and tasks you will meet.

What you will learn

- Smaller, safer images and Dockerfile habits
- Scanning images with Trivy and acting on results
- Restricting registries and tags through admission
- Static analysis of manifests, and signatures

Minimise the image

Every package in an image is something to patch and something an attacker can use. Good habits:

- Use a small base image, such as a minimal or distroless one
- Use multi-stage builds so compilers and build tools do not ship
- Run as a non-root user
- Pin base images by version, ideally by digest
- Do not copy secrets into layers
- Remove shells and package managers from runtime images where practical

A Dockerfile reviewed in a task might use `latest`, run as root and install extra tools. Fix those lines.

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/app

FROM gcr.io/distroless/static:nonroot
COPY --from=build /app /app
USER nonroot
ENTRYPOINT ["/app"]
```

Scan images

Trivy is a common scanner for images, file systems and configuration. It lists known vulnerabilities by severity.

```
trivy image nginx:1.21
trivy image --severity HIGH,CRITICAL nginx:1.21
trivy image --ignore-unfixed --severity CRITICAL nginx:1.27
trivy image -f json -o result.json nginx:1.27
trivy config ./deploy
```

A typical task: scan several images used by pods in a namespace and delete the pods or deployments that use an image with a critical vulnerability, or choose the least vulnerable image. To list images in use:

```
kubectl get pods -n shop -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.containers[*]. \
    image}{"\n"}{end}'
```

Remember that a scan is a snapshot. New vulnerabilities are published every day, so scan again on a schedule and in the pipeline.

Pin by digest

A tag such as `1.27` can point to different content over time. A digest cannot.

```
containers:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  - name: app
    image: nginx@sha256:0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
```

Get the digest with the registry or with `crictl images --digests`. Pinning gives reproducible deployments and prevents a swapped image under the same tag.

Allow only trusted registries

You can block images from unknown places at admission. Options in the curriculum include:

- **ImagePolicyWebhook**, an admission plugin that asks an external service whether an image is allowed
- **OPA Gatekeeper** with a constraint that allows only certain repository prefixes
- **Kyverno** policies that match image names and require digests or signatures

With Gatekeeper, you apply a ConstraintTemplate, then a Constraint naming the namespaces and allowed prefixes. After creating them, test by trying to run an image from another registry and confirm that the API rejects it with the policy message.

```
kubectl get constrainttemplates
kubectl get k8sallowedrepos
kubectl run bad --image=docker.io/library/busybox:1.36 -n shop
```

If you configure ImagePolicyWebhook, you edit the API server manifest to enable the plugin and provide an admission configuration file with a kubeconfig for the webhook. Keep a backup of the manifest, because a mistake here can stop pods from being admitted at all.

Static analysis

Check manifests for risky settings before deploying. Tools such as kubesec score a manifest, and Trivy can scan configuration for misconfigurations, such as privileged containers, missing limits or host paths.

```
kubesec scan pod.yaml
trivy config ./manifests
```

A task may ask you to fix the findings, then re-scan to show improvement. Common fixes: set a security context, drop capabilities, add resource limits and avoid host namespaces.

Signatures and provenance

Signing an image proves who built it and that it has not changed. The Sigstore project provides cosign for signing and verifying, and the SLSA framework describes levels of build integrity. For the exam, know what a signature and an SBOM, the software bill of materials, are for, and how an admission policy can require verified signatures. Also know that your CI system and registry credentials are part of the chain and need the same protection as the cluster.

```
allowed = ("registry.example.com/", "gcr.io/distroless/")
images = ["registry.example.com/shop/web:1.4", "docker.io/library/nginx:latest", "gcr.io/distroles \
    s/static:nonroot"]
for img in images:
    ok = img.startswith(allowed) and not img.endswith(":latest")
    print(("ALLOW " if ok else "DENY  ") + img)
```

Diagram: From source to running image

Part	What it does	What goes wrong
Source and Dockerfile	Minimal, non-root, pinned base.	Latest tags and extra tools.
Build	Multi-stage build, no secrets in layers.	Secrets baked into layers.
Scan and sign	Vulnerability scan and a signature.	Scan results ignored.
Registry	Trusted store with access control.	Public or shared credentials.
Admission	Allowed registries, digests, signatures.	Policy not enforced for all namespaces.
Cluster	Only approved images start.	Unscanned images deployed by hand.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Source and Dockerfile to Build / Build to Scan and sign / Scan and sign to Registry / Registry to Admission / Admission to Cluster

Incident: The image from a lookalike registry

> **Incident:** *A pod started from an image with a name almost identical to an internal one, pulled from a public registry, and ran an unexpected process.*

```
$ kubectl get pod report -n shop -o jsonpath='{.spec.containers[0].image}'
docker.io/shop/report:2.3
$ kubectl get pod report -n shop -o jsonpath='{.metadata.labels}'
{"app":"report"}
(this is a simulated illustration, not a real incident)
```

The manifest had omitted the internal registry prefix, so the runtime pulled a similarly named public image. After this the team required fully qualified image names, added a policy that allowed only the internal registry prefix, and required digests for production namespaces. Testing a deliberately bad image proved that the policy denied it.

Common mistakes

- Treating one scan as permanent
- Allowing the latest tag or short image names
- Enabling an admission plugin without a backup of the API server manifest
- Fixing the Dockerfile but not rebuilding and redeploying the image
- Ignoring the build system and registry as part of the attack surface

Check yourself

Why pin an image by digest?

Answer: A digest identifies exact content and cannot be silently swapped

What does Trivy do?

Answer: Scans images and configuration for known vulnerabilities and misconfigurations

How can a cluster allow images only from a trusted registry?

Answer: An admission policy such as Gatekeeper, Kyverno or ImagePolicyWebhook

What is an SBOM?

Answer: A list of the components inside software

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. CKS: monitoring, runtime security and the mock run

15 minutes. Detect and contain attacks: enable audit logging, write Falco rules, keep containers immutable, investigate suspicious behaviour, then plan a full timed mock run of both exams.

The last CKS domain is about seeing attacks while they happen and after. It is also about 20 percent of the exam. This lesson covers audit logs, runtime detection with Falco, immutability and investigation, and ends with a method for running a full practice exam under time pressure.

What you will learn

- Kubernetes audit logging and how to enable it
- Falco rules and reading alerts
- Immutable containers and spotting suspicious pods
- A plan for a timed mock exam

Audit logging

The API server can record who did what and when. You choose how much to log with an audit policy, which lists rules matched in order. Levels are `None`, `Metadata` (who and what, not the body), `Request` (adds the request body) and `RequestResponse` (adds the response). A small policy:

```
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
  - RequestReceived
rules:
  - level: RequestResponse
    resources:
      - group: ""
        resources: ["secrets"]
  - level: Metadata
    resources:
      - group: ""
        resources: ["pods", "configmaps"]
  - level: None
    users: ["system:kube-proxy"]
  - level: Metadata
```

Rules are evaluated top to bottom and the first match wins, so order matters. Logging secret bodies is sensitive, so most real policies use `Metadata` for secrets. To enable the policy on a kubeadm cluster, edit the API server manifest:

```
cp /etc/kubernetes/manifests/kube-apiserver.yaml /root/kube-apiserver.yaml.bak
vim /etc/kubernetes/manifests/kube-apiserver.yaml
```

Add flags and the volumes that make the files reachable inside the pod:

```
    - --audit-policy-file=/etc/kubernetes/audit/policy.yaml
    - --audit-log-path=/var/log/kubernetes/audit/audit.log
    - --audit-log-maxage=7
    - --audit-log-maxbackup=3
```

Then add `volumeMounts` and `volumes` entries for `/etc/kubernetes/audit` and `/var/log/kubernetes/audit`. Without them the API server cannot read the policy and will not start. After it restarts, search the log:

```
grep '"resource":"secrets"' /var/log/kubernetes/audit/audit.log | tail -n 3
jq 'select(.verb=="delete") | .user.username' /var/log/kubernetes/audit/audit.log | sort | uniq -c
```

MARGA by EknathaLabs marga.eknathalabs.com

Falco

Falco watches system calls on a node and raises alerts when behaviour matches a rule, such as a shell starting inside a container or a sensitive file being read. It runs as a service on each node or as a daemon set. Default rules live in `/etc/falco/falco_rules.yaml`, and your own go in `/etc/falco/falco_rules.local.yaml`, which survives upgrades.

A rule has a name, a description, a condition, an output and a priority:

```
- rule: Shell in container
  desc: A shell was started inside a container
  condition: evt.type = execve and container.id != host and proc.name in (bash, sh)
  output: "Shell started in container (user=%user.name container=%container.name image=%container. \
    image.repository command=%proc.cmdline)"
  priority: WARNING
```

Read alerts with `journalctl -u falco -f` or from the configured output file. A task might say: find the pod that spawned a shell in the last few minutes, or change a rule's output format, then confirm that an alert shows the new format after triggering it with `kubectl exec`. After editing rules, restart the service and check for syntax errors in its log.

Immutable containers

Attackers install tools and change files after getting in. If the root filesystem is read-only and containers do not run privileged, much of that fails. Review a namespace for containers that break the rule:

```
kubectl get pods -n shop -o json | jq -r '.items[] | select(.spec.containers[].securityContext.pri \
    vileged==true) | .metadata.name'
kubectl get pods -n shop -o json | jq -r '.items[] | select(.spec.containers[].securityContext.rea \
    dOnlyRootFilesystem != true) | .metadata.name'
```

Fix by setting `readOnlyRootFilesystem: true` and adding writable `emptyDir` mounts for specific paths. Do not use `hostPath`, host networking or host PID unless the task justifies it.

Investigate a suspicious pod

A method that works: identify the pod and node, check what it runs and as whom, review recent audit events, check Falco alerts, inspect the image, and then contain it by isolating with a network policy, scaling down the owner and preserving evidence before deleting. Do not delete first; you lose the evidence.

```
kubectl get pod odd -n shop -o yaml | grep -E "image:|serviceAccount|hostPath|privileged"
kubectl logs odd -n shop
kubectl exec odd -n shop -- ps aux
```

Plan a full mock run

Practise under real conditions, because the second half of the exam is where unpractised candidates slip.

1. **Set up.** Start a timer for the real duration. Use only the permitted docs.
2. **Triage first.** Spend three minutes listing the tasks with weights and rough effort.
3. **Quick points first.** Do short, high-weight tasks, then medium ones.
4. **Flag at the time cap.** If a task takes more than about twice its expected time, flag it.
5. **Verify each result.** Run the check command before moving on.
6. **Reserve the last ten minutes.** Return to flagged tasks and re-read their wording.
7. **Review afterwards.** For each missed task, write what you would do next time, and drill it.

Run at least two full mocks per exam, a week apart, and compare timings per domain.

```
mock = [
    ("etcd backup", 7, 8, 7),
    ("fix kubelet", 8, 12, 15),
    ("network policy", 5, 7, 6),
    ("pod security", 6, 9, 9),
]
```

MARGA by EknathaLabs marga.eknathalabs.com

```
for name, weight, planned, actual in mock:
    flag = "over" if actual > planned * 1.5 else "ok"
    print(f"{name:16} weight {weight}  planned {planned}  actual {actual}  {flag}")
print("score weight done:", sum(w for _, w, _, _ in mock))
```

Diagram: Seeing and containing an attack

Part	What it does	What goes wrong
Audit log	Who did what through the API.	Policy logging nothing useful.
Falco alerts	Suspicious behaviour on nodes.	Rules never tested.
Investigate	Pod, node, process, image and account.	Deleting before preserving evidence.
Contain	Isolate with policy, scale down owner.	Slow or partial isolation.
Immutable fix	Read-only root and no privilege.	The same weakness left in place.
Mock run review	Timings, misses and drills.	Never practising under time.

Flow: Audit log to Investigate / Falco alerts to Investigate / Investigate to Contain / Contain to Immutable fix / Immutable fix to Mock run review

Incident: The audit policy that broke the API server

Incident: *A candidate enabled audit logging, and the API server never came back.*

```
$ crictl ps -a | grep kube-apiserver
a91c3e  kube-apiserver  Exited  3 minutes ago
$ crictl logs a91c3e 2>&1 | tail -n 2
Error: failed to load audit policy file: open /etc/kubernetes/audit/policy.yaml: no such file or d \
    irectory
(this is a simulated illustration, not a real incident)
```

The flags were added but the audit directory was not mounted into the API server pod, so the process could not read the policy file. Adding the `volumes` and `volumeMounts` entries, and restoring the backup first to get kubectl working again, fixed it. This is why you copy the manifest before editing and check the container log if the pod does not return.

Common mistakes

- Adding audit flags without the volume mounts
- Logging secret bodies at RequestResponse level in real clusters
- Editing the default Falco rules file instead of the local one
- Deleting a suspicious pod before looking at it
- Doing only single tasks in practice and never a full timed run

Check yourself

Which audit level records the request body but not the response?

Answer: Request

Where should custom Falco rules go?

Answer: In the local rules file, which survives upgrades

Why use a read-only root filesystem?

Answer: It stops attackers installing tools or changing files in the container

What is the safest first move on a suspicious pod?

Answer: Investigate and preserve evidence, then contain

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: CKA and CKS exam

Exam setup, imperative commands, etcd backup, kubeadm upgrade, RBAC checks, scheduling and a hardening checklist.

Setup in the first two minutes

Step	Command
Alias	`alias k=kubectl`
Completion	`source <(kubectl completion bash)` and `complete -F __start_kubectl k`
Dry run variable	`export do="--dry-run=client -o yaml"`
Context	Paste the command given in the task before every task

Imperative commands

Job	Command
Pod	`k run web --image=nginx`
Deployment	`k create deploy web --image=nginx --replicas=3`
Expose	`k expose deploy web --port=80`
Role	`k create role r --verb=get,list --resource=pods`
Binding	`k create rolebinding rb --role=r --user=jane`
Field names	`k explain pod.spec.containers`

etcd backup and restore

Step	Command		
Find paths	`grep -E "cert	key	listen-client" /etc/kubernetes/manifests/etcd.yaml`
Save	`ETCDCTL_API=3 etcdctl snapshot save FILE --endpoints=... --cacert=... --cert=... --key=...`		
Verify	`etcdctl snapshot status FILE`		
Restore	`etcdctl snapshot restore FILE --data-dir=/var/lib/etcd-new`		
Switch	Edit the hostPath of the data volume in the etcd manifest		

kubeadm upgrade order

Where	Steps
Control plane	upgrade kubeadm, plan, apply, drain, upgrade kubelet, restart, uncordon
Each worker	drain, upgrade kubeadm, upgrade node, upgrade kubelet, restart, uncordon

MARGA by EknathaLabs marga.eknathalabs.com

Permission checks

Question	Command
Can this user do it?	`k auth can-i get pods/log -n dev --as jane`
What can I do?	`k auth can-i --list`
Who is bound?	`k get rolebinding,clusterrolebinding -A -o wide`

Scheduling quick map

Need	Use
Pick a node by label	nodeSelector or node affinity
Keep pods off a node	Taint on the node
Allow a tainted node	Toleration on the pod
Spread replicas	Topology spread constraints

Hardening checklist

Area	Control
API and kubelet	Anonymous access off, Webhook authorization
Network	Default deny, allow DNS
Pods	Non-root, drop capabilities, no privilege escalation
Host	AppArmor, seccomp RuntimeDefault
Data	Encryption at rest, rotate after exposure
Images	Scan with Trivy, pin digests, allow only your registry
Detection	Audit policy and Falco rules

MARGA by EknathaLabs
marga.eknathalabs.com